# Vídeo 5.3: Validando campos e devolvendo erros claros

**Duração-alvo:** 7 min | **Conceitos:** validação, código 400, erro padronizado e tratador de erros | **O aluno sai sabendo:** barrar dados inválidos com uma mensagem clara e padronizar todas as respostas de erro.

**Antes de gravar:** partir do código do vídeo 5.2, com os três pedidos ruins do vídeo 5.1 prontos no requisicoes.http.

**[0:00] Abertura**

*Na tela:* título do vídeo.

*Fala:* “Hora de colocar o porteiro para conferir as fichas. Neste vídeo vamos escrever a validação do livro, devolver erros claros e padronizados e cuidar de dois casos que ainda estão sem resposta boa: o imprevisto e o endereço que não existe.”

**[0:40] As regras**

*Na tela:* uma lista com as duas regras.

*Fala:* “Primeiro, decidir as regras. A nossa ficha de livro precisa de título e autor, os dois textos, e nenhum pode estar vazio ou só com espaços. Se alguma regra falhar, respondemos 400 com uma lista dizendo o que está errado.”

**[1:20] Escrevendo o porteiro da ficha**

*Na tela:* no server.js, acima das rotas, escrever a função abaixo.

```
function validarLivro(req, res, next) {
  const { titulo, autor } = req.body ?? {};
  const erros = [];

  if (typeof titulo !== 'string' || titulo.trim() === '') {
        erros.push('O título é obrigatório e deve ser um texto.');
  }

  if (typeof autor !== 'string' || autor.trim() === '') {
        erros.push('O autor é obrigatório e deve ser um texto.');
  }

  if (erros.length > 0) {
        return res.status(400).json({ erro: 'Dados inválidos', detalhes: erros });
  }

  next();
}
```

*Fala:* “Escrevo o middleware validarLivro. Pego título e autor do corpo. Se por algum motivo não veio corpo nenhum, esses dois pontos de interrogação seguidos dizem: nesse caso, use um objeto vazio. Crio uma lista de erros, começando vazia. Para cada regra: se o título não é um texto, ou se, tirando os espaços das pontas, está vazio, adiciono uma mensagem

à lista. Faço o mesmo com o autor. Depois, se a lista de erros tem alguma coisa, barro: respondo 400, com um JSON padronizado, que tem um campo erro com o resumo e um campo detalhes com a lista de mensagens. E return, para parar aqui. Se não há erros, next: pode passar."

**[3:20] Colocando nas rotas**

*Na tela:* acrescentar validarLivro entre o endereço e a função, nas rotas POST e PUT. O restante das rotas não muda.

```
app.post('/livros', validarLivro, async (req, res) => {
  // ... igual ao anterior
});

app.put('/livros/:id', validarLivro, async (req, res) => {
  // ... igual ao anterior
});
```

*Fala:* "Para usar, colocamos validarLivro entre o endereço e a função da rota, tanto no POST quanto no PUT. Assim, a conferência é escrita uma vez e vale para as duas. A rota só executa se o porteiro deixou passar. Nada dentro das rotas muda."

**[4:00] Testando**

*Na tela:* reiniciar o servidor, repetir os três pedidos ruins do vídeo 5.1 e um pedido correto.

*Fala:* "Repito os três pedidos ruins. O sem autor agora devolve 400, com a mensagem dizendo que o autor é obrigatório. O título vazio também: 400. O título numérico: 400, com a mensagem de que deve ser texto. E um pedido correto continua dando 201. Os erros agora ajudam quem usa a API: dizem o que corrigir."

**[5:00] Os dois casos que faltam**

*Na tela:* no fim do arquivo, depois de todas as rotas e antes de iniciar(), escrever os dois blocos abaixo. Depois, pedir /estante-que-nao-existe no navegador.

```
app.use((req, res) => {
  res.status(404).json({ erro: 'Rota não encontrada' });
});

app.use((erro, req, res, next) => {
  console.error(erro);
  res.status(500).json({ erro: 'Erro interno do servidor' });
});
```

*Fala:* "Faltam dois casos. O primeiro: lembra que, quando pedimos um endereço que não existe, aparecia aquela mensagem em formato diferente, cannot GET? Vamos padronizar. No fim do arquivo, depois de todas as rotas, adicionamos um middleware que responde 404 no mesmo formato JSON. Ele só é alcançado se nenhuma rota atendeu o pedido. O segundo: o imprevisto. Um middleware com quatro nomes, começando por erro, é especial: o Express o chama quando algo dá errado de verdade no servidor. Nele, registramos o problema no terminal, para a gente ver, e respondemos ao cliente com 500 e uma mensagem genérica. Nunca mostramos ao cliente os detalhes técnicos do erro: eles podem revelar como o sistema funciona por dentro."

**[6:20] Resumo**

*Na tela:* o formato padrão de erro, { "erro": "..." }.

*Fala:* “Resumindo: um middleware de validação confere os dados antes da rota e responde 400 com uma mensagem clara. Todo erro da nossa API agora tem o mesmo formato, um JSON com o campo erro. E o tratador do final cuida do endereço que não existe e do imprevisto. No próximo vídeo, vamos arrumar a casa: o código está crescendo, e vamos separá-lo em partes.”

**Exercício sugerido:** acrescentar uma regra ao validarLivro (por exemplo, o título ter no máximo 120 caracteres) e testar com um pedido que viole a regra.